

Woxsen University

School of Technology



A

PROJECT REPORT

on

LIBRARY CIRCULATION & DIGITAL RESOURCE MANAGEMENT SYSTEM

Submitted in partial fulfillment of the requirements for the degree of

B.Tech in Artificial Intelligence & Machine Learning

Submitted by:

Rohan Shikhar

25WU0101112

Department of Computer Science & Engineering
Section: AIML Whales
Course: Database Management Systems

Academic Year 2025 - 2026

Table of Contents

01	Abstract	
02	Introduction & Problem Statement	
03	Objectives & Scope	
04	Software & Hardware Requirements	
05	ER Diagram	
06	Relational Schema & Normalization	
07	Data Dictionary	
08	SQL Commands (DDL & DML) with Sample Outputs	
09	Queries with Outputs	
10	UI Design & Screenshots	
11	Implementation Details	
12	Testing	
13	Conclusion & Future Enhancements	
14	References	
15	Appendix: GitHub Repository Link	

This project presents the design, development, and implementation of a **Library Circulation and Digital Resource Management System** using MySQL 9.7 as the relational database engine and Python Flask as the web application backend. The system models a real-world university library through a normalized 13-table relational schema in Third Normal Form (3NF), managing the full lifecycle of physical book copies — from cataloguing and member registration to circulation (issue and return), fine calculation, and reservation queuing — while simultaneously indexing digital e-books and journals.

A browser-based user interface, built with Vanilla HTML, CSS, and JavaScript, connects to the MySQL database via a REST API and supports live insertion, deletion, and viewing of records. The project demonstrates practical application of relational modelling concepts including entity identification, foreign-key referencing, many-to-many junction tables, and multi-table analytical queries.

Background

University libraries are complex operational environments that maintain thousands of physical books, track borrowing by hundreds of members, impose return deadlines, collect fines for overdue items, and host growing collections of electronic resources. Managing this with paper registers or spreadsheets leads to data inconsistency, duplication, and slow lookups.

A relational database provides the ideal solution: structured tables with referential integrity ensure no book copy can be issued to a non-existent member, no fine can reference a non-existent issue, and aggregation queries can instantly answer complex operational questions.

Problem Statement

The system must solve the following real-world problems faced by a university library:

- **Book catalogue management:** Maintaining book titles, ISBNs, authors, categories, and publishers without redundancy.
- **Physical copy tracking:** Each physical copy has a unique accession number and availability status (Available / Issued / Reserved / Lost).
- **Circulation control:** Recording issue and return dates per copy per member, preventing double-issuing.
- **Overdue and fine management:** Identifying overdue books and associating fine amounts with issue records.
- **Reservation queuing:** Allowing members to reserve popular titles and tracking queue positions.
- **Digital resource access logging:** Tracking who accessed which e-book or journal and when.
- **Role-based portal access:** Separate workflows for Admin (insert, delete, manage) and Student (browse, borrow).

Objectives

1. Design a normalized relational schema (3NF) that eliminates data redundancy and maintains referential integrity for all library entities.
2. Implement the full DDL (table creation, constraints, views) and DML (seed data, transactions) in MySQL 9.7.
3. Build a Flask REST API that exposes CRUD endpoints for all major entities.
4. Develop a browser-based UI supporting live Insert, Delete, and View operations with real-time MySQL synchronization.
5. Execute and present five analytical SQL queries covering availability tracking, circulation history, overdue reporting, demand analysis, and fine summaries.
6. Demonstrate the system live during Presentation-III.

Scope

In Scope: Book catalogue, physical copy management, member registration, issue/return workflow, fine records, reservation queue, digital resources, digital access log, role-based UI, 5 evaluated SQL queries, REST API with JSON responses.

Out of Scope: Payment gateway integration, email/SMS notifications, mobile application, barcode/RFID hardware, multi-branch library support.

04 Software & Hardware Requirements

Software Requirements

Component	Technology / Version	Role
Database Engine	MySQL Community Server 9.7 (InnoDB)	Persistent relational data store
Backend Language	Python 3.14	Application logic and API server
Web Framework	Flask 3.x	REST API routing and Jinja2 templating
Database Connector	PyMySQL 1.x	Python-to-MySQL communication
Frontend	HTML5 + Vanilla CSS + Vanilla JS	User interface rendered in browser
Fonts	Google Fonts (Inter, Space Grotesk, JetBrains Mono)	UI typography
Operating System	Windows 11	Development and runtime environment
Version Control	Git + GitHub	Source code management

Hardware Requirements

Component	Minimum	Used in Project
Processor	Dual-core, 1.5 GHz	Intel/AMD multi-core laptop
RAM	4 GB	8+ GB
Storage	500 MB free	SSD
Network	localhost (127.0.0.1)	LAN / localhost:5000
Browser	Any modern browser	Google Chrome / Edge

05 ER Diagram

The Entity-Relationship diagram below shows all 13 entities and their relationships. Crow's-foot notation denotes cardinality.

```
CATEGORY (1) — (M) BOOK (M) — (M) AUTHOR
                                     |
                                     | 1:M
                                     | (via BOOK_AUTHOR)
PUBLISHER (1) — (M) BOOK
                                     |
                                     | (M) COPY — Status: Available / Issued / Reserved / Lost
                                     |
                                     | 1:M (CopyID)
MEMBER (1) — (M) ISSUE — (0..1) RETURN
|
|
|
|
| — (M) RESERVATION — (M) BOOK
|
| — (M) DIGITAL_ACCESS_LOG — (M) DIGITAL_RESOURCE
|
| — (0..1) FINE
```

Entity Summary

Entity	Primary Key	Description
CATEGORY	CategoryID	Subject classification (DBMS, OS, AI, etc.)
PUBLISHER	PublisherID	Book publisher details
AUTHOR	AuthorID	Author name and country
BOOK	BookID	Unique book title entity
BOOK_AUTHOR	(BookID, AuthorID)	M:N junction - one book can have many authors
COPY	CopyID	Individual physical copy on the shelf
MEMBER	MemberID	Registered student or faculty
ISSUE	IssueID	Circulation transaction (who borrowed which copy)
RETURN	ReturnID	Closes an open issue record
RESERVATION	ReservationID	Queue entry when all copies are issued
FINE	FineID	Overdue late fee linked to an issue
DIGITAL_RESOURCE	ResourceID	E-book or journal with access URL
DIGITAL_ACCESS_LOG	LogID	Audit log of digital resource access events

Relational Schema

```
CATEGORY      (CategoryID PK, CategoryName UNIQUE NOT NULL)
PUBLISHER      (PublisherID PK, PublisherName UNIQUE NOT NULL, City)
AUTHOR        (AuthorID PK, Name NOT NULL, Country)
BOOK          (BookID PK, ISBN UNIQUE NOT NULL, Title NOT NULL,
               CategoryID FK→CATEGORY, PublisherID FK→PUBLISHER)
BOOK_AUTHOR    (BookID FK→BOOK, AuthorID FK→AUTHOR) [Composite PK]
COPY          (CopyID PK, AccessionNo UNIQUE NOT NULL,
               BookID FK→BOOK, Status ENUM[Available,Issued,Reserved,Lost])
MEMBER        (MemberID PK, Name NOT NULL, Email UNIQUE NOT NULL, MemberType)
ISSUE         (IssueID PK, MemberID FK→MEMBER, CopyID FK→COPY,
               IssueDate DATE, DueDate DATE)
RETURN        (ReturnID PK, IssueID UNIQUE FK→ISSUE, ReturnDate DATE)
RESERVATION    (ReservationID PK, MemberID FK→MEMBER, BookID FK→BOOK,
               ReservationDate DATE, QueueNo INT, Status VARCHAR)
FINE          (FineID PK, IssueID UNIQUE FK→ISSUE,
               Amount DECIMAL(8,2), PaidStatus ENUM[Paid,Unpaid])
DIGITAL_RESOURCE (ResourceID PK, Title, Format, AccessURL)
DIGITAL_ACCESS_LOG (LogID PK, MemberID FK→MEMBER,
                   ResourceID FK→DIGITAL_RESOURCE, AccessDate DATETIME)
```

Normalization Analysis

First Normal Form (1NF)

All attributes are atomic. No repeating groups. The M:N relationship between BOOK and AUTHOR is resolved into the `BOOK_AUTHOR` junction table instead of storing a comma-delimited author column.

Second Normal Form (2NF)

Every non-key attribute is fully functionally dependent on the entire primary key. The only composite PK is `BOOK_AUTHOR(BookID, AuthorID)` which has no non-key attributes - 2NF is satisfied trivially.

Third Normal Form (3NF)

No transitive dependencies exist:

- Publisher city is in `PUBLISHER`, not in `BOOK` - removing the transitive dependency `Book->Publisher->City`.
- Category name is in `CATEGORY`, not repeated in `BOOK`.
- Author country is in `AUTHOR`, not in `BOOK_AUTHOR`.
- Fine amount is in `FINE`, not duplicated in `MEMBER` or `COPY`.

The schema is fully in 3NF. All determinants in non-trivial functional dependencies are superkeys of their respective tables.

CATEGORY

Column	Type	Constraints
CategoryID	INT	PK, AUTO_INCREMENT
CategoryName	VARCHAR(100)	NOT NULL, UNIQUE

PUBLISHER

Column	Type	Constraints
PublisherID	INT	PK, AUTO_INCREMENT
PublisherName	VARCHAR(150)	NOT NULL, UNIQUE
City	VARCHAR(100)	-

AUTHOR

Column	Type	Constraints
AuthorID	INT	PK, AUTO_INCREMENT
Name	VARCHAR(150)	NOT NULL
Country	VARCHAR(100)	-

BOOK

Column	Type	Constraints
BookID	INT	PK
ISBN	VARCHAR(20)	NOT NULL, UNIQUE
Title	VARCHAR(200)	NOT NULL
CategoryID	INT	FK → CATEGORY
PublisherID	INT	FK → PUBLISHER

BOOK_AUTHOR (Junction)

Column	Type	Constraints
BookID	INT	PK, FK → BOOK
AuthorID	INT	PK, FK → AUTHOR

COPY

Column	Type	Constraints
CopyID	INT	PK, AUTO_INCREMENT
AccessionNo	VARCHAR(50)	NOT NULL, UNIQUE
BookID	INT	FK → BOOK
Status	ENUM	Available/Issued/Reserved

MEMBER

Column	Type	Constraints
MemberID	INT	PK, AUTO_INCREMENT
Name	VARCHAR(150)	NOT NULL
Email	VARCHAR(150)	NOT NULL, UNIQUE
MemberType	VARCHAR(50)	DEFAULT 'Student'

ISSUE

Column	Type	Constraints
IssueID	INT	PK, AUTO_INCREMENT
MemberID	INT	FK → MEMBER
CopyID	INT	FK → COPY
IssueDate	DATE	NOT NULL
DueDate	DATE	NOT NULL

RETURN

Column	Type	Constraints
ReturnID	INT	PK, AUTO_INCREMENT
IssueID	INT	UNIQUE FK → ISSUE
ReturnDate	DATE	NOT NULL

FINE

Column	Type	Constraints
FineID	INT	PK, AUTO_INCREMENT
IssueID	INT	UNIQUE FK → ISSUE
Amount	DECIMAL(8,2)	NOT NULL
PaidStatus	ENUM	Paid / Unpaid

RESERVATION		
Column	Type	Constraints
ReservationID	INT	PK, AUTO_INCREMENT
MemberID	INT	FK → MEMBER
BookID	INT	FK → BOOK
ReservationDate	DATE	NOT NULL
QueueNo	INT	DEFAULT 1
Status	VARCHAR(50)	DEFAULT 'Pending'

DIGITAL_RESOURCE		
Column	Type	Constraints
ResourceID	INT	PK, AUTO_INCREMENT
Title	VARCHAR(200)	NOT NULL
Format	VARCHAR(50)	NOT NULL
AccessURL	VARCHAR(255)	NOT NULL

DIGITAL_ACCESS_LOG		
Column	Type	Constraints
LogID	INT	PK, AUTO_INCREMENT
MemberID	INT	FK → MEMBER
ResourceID	INT	FK → DIGITAL_RESOURCE
AccessDate	DATETIME	DEFAULT CURRENT_TIMESTAMP

DDL - Database and Table Creation

```
CREATE DATABASE LibraryDBMS;
USE LibraryDBMS;

-- CATEGORY table
CREATE TABLE CATEGORY (
    CategoryID INT PRIMARY KEY AUTO_INCREMENT,
    CategoryName VARCHAR(100) NOT NULL UNIQUE
) ENGINE=InnoDB;

-- BOOK table with FK references
CREATE TABLE BOOK (
    BookID INT PRIMARY KEY,
    ISBN VARCHAR(20) NOT NULL UNIQUE,
    Title VARCHAR(200) NOT NULL,
    CategoryID INT NOT NULL,
    PublisherID INT NOT NULL,
    CONSTRAINT fk_book_category FOREIGN KEY (CategoryID)
        REFERENCES CATEGORY(CategoryID) ON UPDATE CASCADE,
    CONSTRAINT fk_book_publisher FOREIGN KEY (PublisherID)
        REFERENCES PUBLISHER(PublisherID) ON UPDATE CASCADE
) ENGINE=InnoDB;

-- BOOK_AUTHOR junction table (M:N)
CREATE TABLE BOOK_AUTHOR (
    BookID INT NOT NULL,
    AuthorID INT NOT NULL,
    PRIMARY KEY (BookID, AuthorID),
    CONSTRAINT fk_ba_book FOREIGN KEY (BookID)
        REFERENCES BOOK(BookID) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_ba_author FOREIGN KEY (AuthorID)
        REFERENCES AUTHOR(AuthorID) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

-- ISSUE table
CREATE TABLE ISSUE (
    IssueID INT PRIMARY KEY AUTO_INCREMENT,
    MemberID INT NOT NULL,
    CopyID INT NOT NULL,
    IssueDate DATE NOT NULL,
    DueDate DATE NOT NULL,
    CONSTRAINT fk_issue_member FOREIGN KEY (MemberID)
        REFERENCES MEMBER(MemberID) ON DELETE CASCADE,
    CONSTRAINT fk_issue_copy FOREIGN KEY (CopyID)
        REFERENCES COPY(CopyID) ON DELETE CASCADE
) ENGINE=InnoDB;
```

DML - INSERT Samples

```
-- Insert categories
INSERT INTO CATEGORY (CategoryID, CategoryName) VALUES
(1, 'DBMS'), (2, 'OS'), (3, 'Networking'), (4, 'Programming'),
(5, 'DSA'), (6, 'AI'), (7, 'Cloud'), (8, 'SE');

-- Insert a book
INSERT INTO BOOK (BookID, ISBN, Title, CategoryID, PublisherID) VALUES
(201, '978-0073523323', 'Database Management System', 1, 1);

-- Issue a book copy to a member
INSERT INTO ISSUE (MemberID, CopyID, IssueDate, DueDate) VALUES
(1, 1, '2025-09-01', '2025-09-15');
```

Sample Output - SHOW TABLES:

```
mysql> SHOW TABLES;
+-----+
| Tables_in_LibraryDBMS |
+-----+
| AUTHOR                 |
| BOOK                   |
| BOOK_AUTHOR            |
| CATEGORY               |
| COPY                   |
| DIGITAL_ACCESS_LOG     |
| DIGITAL_RESOURCE       |
| FINE                   |
| ISSUE                  |
| MEMBER                 |
| PUBLISHER              |
| RESERVATION            |
| RETURN                 |
+-----+
13 rows in set (0.01 sec)
```

DML - DELETE Sample

```
-- Delete a book (cascades to COPY, BOOK_AUTHOR, ISSUE, FINE, RETURN)
DELETE FROM BOOK WHERE BookID = 221;

-- Record a book return
INSERT INTO `RETURN` (IssueID, ReturnDate)
VALUES (3, CURDATE());
```

Books View (deduplicating author names)

```
CREATE OR REPLACE VIEW Books AS
SELECT
    b.BookID AS Book_ID,  b.Title,
    IFNULL(GROUP_CONCAT(DISTINCT a.Name SEPARATOR ', '), 'Unknown') AS Author,
    c.CategoryName AS Category,
    COUNT(DISTINCT CASE WHEN cp.Status = 'Available' THEN cp.CopyID END) AS Available_Copies
FROM BOOK b
JOIN CATEGORY c ON b.CategoryID = c.CategoryID
LEFT JOIN BOOK_AUTHOR ba ON b.BookID = ba.BookID
LEFT JOIN AUTHOR a ON ba.AuthorID = a.AuthorID
LEFT JOIN COPY cp ON b.BookID = cp.BookID
GROUP BY b.BookID, b.Title, c.CategoryName;
```

09 Queries with Outputs

Query 1 - Books Available on the Shelf

```
SELECT * FROM Books WHERE Available_Copies > 0;
```

```
+-----+-----+-----+-----+
| Book_ID| Title                | Author          | Category   | Available_Copies |
+-----+-----+-----+-----+
| 201 | Database Management System| Korth          | DBMS       | 5 |
| 202 | Operating Systems        | Galvin         | OS         | 4 |
| 203 | Computer Networks        | Forouzan       | Networking | 6 |
| 204 | Java Programming         | Herbert Schildt| Programming| 7 |
| 205 | Python Basics           | John Zelle     | Programming| 8 |
| ... | ...                     | ...            | ...        | ... |
+-----+-----+-----+-----+
19 rows in set (0.04 sec)
```

Query 2 - Circulation Tracking

```
SELECT m.Member_Name, b.Title, i.Issue_Date, i.Return_Date
FROM Issued_Books i
JOIN Members m ON i.Member_ID = m.Member_ID
JOIN Books b ON i.Book_ID = b.Book_ID;
```

```
+-----+-----+-----+-----+
| Member_Name | Title                | Issue_Date | Return_Date |
+-----+-----+-----+-----+
| Rohan Shikhar | Database Management System| 2025-09-01 | NULL        |
| Sarthak Sharma | Operating Systems        | 2025-09-02 | 2025-09-18 |
| Soumya Singh | Artificial Intelligence   | 2025-09-03 | NULL        |
| Shreshtha Gupta | Java Programming         | 2025-09-04 | 2025-09-19 |
| Aman Verma    | Machine Learning         | 2025-09-05 | NULL        |
+-----+-----+-----+-----+
10 rows in set (0.03 sec)
```

Query 3 - Overdue Books

```
SELECT m.Member_Name, b.Title, i.Due_Date
FROM Issued_Books i
JOIN Members m ON i.Member_ID = m.Member_ID
JOIN Books b ON i.Book_ID = b.Book_ID
WHERE i.Return_Date IS NULL AND i.Due_Date < CURDATE();
```

```

+-----+-----+-----+
| Member_Name | Title | Due_Date |
+-----+-----+-----+
| Rohan Shikhar | Database Management System | 2025-09-15 |
| Soumya Singh | Artificial Intelligence | 2025-09-17 |
| Aman Verma | Machine Learning | 2025-09-19 |
| Rahul Kumar | Python Basics | 2025-09-21 |
| Arjun Mehta | Data Mining | 2025-09-23 |
| Ananya Jain | IoT Fundamentals | 2025-09-24 |
+-----+-----+-----+
6 rows in set (0.02 sec)

```

Query 4 - Demand Analysis (Most Issued Titles)

```

SELECT b.Title, COUNT(*) AS Times_Issued
FROM Issued_Books i
JOIN Books b ON i.Book_ID = b.Book_ID
GROUP BY b.Book_ID, b.Title
ORDER BY Times_Issued DESC;

```

```

+-----+-----+
| Title | Times_Issued |
+-----+-----+
| Database Management System | 1 |
| Operating Systems | 1 |
| Artificial Intelligence | 1 |
| ... | ... |
+-----+-----+
10 rows in set (0.02 sec)

```

Query 5 - Member Fine Summary

```

SELECT m.Member_Name, m.Member_Type,
       COUNT(i.Issue_ID) AS Total_Issued,
       IFNULL(SUM(f.Fine_Amount),0) AS Total_Fine
FROM Members m
LEFT JOIN Issued_Books i ON m.Member_ID = i.Member_ID
LEFT JOIN Fines f ON i.Issue_ID = f.Issue_ID
GROUP BY m.Member_ID, m.Member_Name, m.Member_Type
ORDER BY Total_Fine DESC;

```

```

+-----+-----+-----+-----+
| Member_Name | Member_Type | Total_Issued | Total_Fine |
+-----+-----+-----+-----+
| Rohan Shikhar | Student | 1 | 50.00 |
| Soumya Singh | Student | 1 | 25.00 |
| Aman Verma | Student | 1 | 15.00 |
| Rahul Kumar | Student | 1 | 10.00 |

```

Arjun Mehta	Student		1		10.00	
Sarthak Sharma	Student		1		0.00	
Shreshtha Gupta	Student		1		0.00	
Priya Patel	Student		1		0.00	
Sneha Reddy	Student		1		0.00	
Ananya Jain	Student		1		0.00	

+-----+-----+-----+-----+

10 rows in set (0.03 sec)

10 UI Design & Screenshots

Design Philosophy

The UI follows a **Monochrome Bento Grid** design language: dark backgrounds on the login screen, clean white card panels in the dashboard, and high-contrast typography. No emojis are used - SVG icons and text-based status badges replace them. The design is role-aware; Admins see full CRUD controls while Students see a browse-and-borrow interface.

Screens and Operations

Screen	Description	Operations
Login Page	Role selection (Admin / Student) with credentials	-
Dashboard	Stats: total books, copies, members, active loans, unpaid fines	View
Books	Full book catalogue with author, category, publisher, copy count	View, Insert, Delete
Members	Registered library members list	View, Insert, Delete
Issue / Return	Circulation table - issue a book, record a return	View, Insert
Fines	Outstanding fine records linked to issue IDs	View
Digital Resources	E-book and journal catalogue with access links	View
SQL Explorer	Run the 5 evaluated queries directly from the portal	View

Frontend Technology

- **HTML5** - semantic markup, single-page layout with dynamic section switching via JavaScript
- **Vanilla CSS** - CSS Grid, CSS custom properties, keyframe animations, glassmorphism effects
- **Vanilla JavaScript** - Fetch API for REST calls, DOM manipulation, role-based conditional rendering
- **Google Fonts** - Inter (body text), Space Grotesk (headings), JetBrains Mono (code and badges)

Note: Live screenshots are captured during the Presentation-III demo and uploaded to the Presentation-III/ folder on GitHub.

11 Implementation Details

Technology Stack

Layer	Technology	Responsibility
Database	MySQL 9.7 (InnoDB)	13-table 3NF schema, FK constraints, ENUM fields, views
DB Connector	PyMySQL 1.x	Connection management, parameterized queries
API Server	Flask 3.x (Python 3.14)	REST routing, JSON serialization, CORS headers
Frontend	HTML5 + CSS + JS	Single-page app, Fetch API, dynamic DOM rendering

Database Connectivity

```
import pymysql
from config import Config

def get_connection():
    return pymysql.connect(
        host      = Config.MYSQL_HOST,
        port      = Config.MYSQL_PORT,
        user      = Config.MYSQL_USER,
        password  = Config.MYSQL_PASSWORD,
        database  = Config.MYSQL_DB,
        cursorclass = pymysql.cursors.DictCursor,
        autocommit = True
    )

def query_db(sql, args=(), one=False):
    conn = get_connection()
    try:
        with conn.cursor() as cur:
            cur.execute(sql, args)
            return cur.fetchone() if one else cur.fetchall()
    finally:
        conn.close()
```

REST API Endpoint Map

Method	Endpoint	Description
GET	/api/status	DB connection status and dashboard stats
GET / POST / DELETE	/api/books[/<id>]	Fetch all / insert / delete books
GET / POST / DELETE	/api/members[/<id>]	Fetch all / register / remove members
GET / POST	/api/issues	Fetch all issues / issue a copy
POST	/api/returns	Record a book return
GET	/api/digital-resources	Fetch digital resource catalogue
GET	/api/query/<n>	Run evaluated query 1-5, return JSON

Issue Book Endpoint (key snippet)

```
@app.route('/api/issues', methods=['POST'])
def api_issue_book():
    data = request.json or {}
    member_id = data.get('member_id')
    copy_id = data.get('copy_id')
    due_date = data.get('due_date')

    # Guard: verify copy is Available
    copy = query_db(
        "SELECT Status FROM COPY WHERE CopyID=%s", (copy_id,), one=True)
    if not copy or copy['Status'] != 'Available':
        return jsonify({"success": False, "error": "Copy not available"}), 400

    issue_date = datetime.date.today().isoformat()
    modify_db(
        "INSERT INTO ISSUE (MemberID,CopyID,IssueDate,DueDate) VALUES (%s,%s,%s,%s)",
        (member_id, copy_id, issue_date, due_date)
    )
    modify_db("UPDATE COPY SET Status='Issued' WHERE CopyID=%s", (copy_id,))
    return jsonify({"success": True})
```

12 Testing

#	Test Case	Input / Action	Expected Result	Actual Result	Status
1	Insert new book	POST /api/books (valid data)	201 created; visible in list	Inserted and reflected in UI	PASS
2	Delete existing book	DELETE /api/books/221	200 OK; book removed	Deleted; copies cascade-removed	PASS
3	Delete non-existent book	DELETE /api/books/9999	404 Not Found	404 returned with error	PASS
4	Issue Available copy	POST /api/issues (CopyID=2)	Issue created; status Issued	ISSUE row inserted; status updated	PASS
5	Issue already-Issued copy	POST /api/issues (CopyID=1)	400 - "Copy not available"	Error returned; UI alert shown	PASS
6	Record a return	POST /api/returns (IssueID=3)	RETURN row; copy Available	Return recorded; status reverted	PASS
7	Insert duplicate ISBN	POST /api/books (existing ISBN)	UNIQUE error 400	Error returned; no duplicate	PASS
8	Author deduplication	GET /api/books	Author once per book	GROUP_CONCAT(DISTINCT) works	PASS
9	Dashboard stats	GET /api/status after insert	Count increments	Count correct	PASS
10	Query 5 (fine summary)	GET /api/query/5	Rohan at top with Rs.50	Output matched exactly	PASS
11	Role-based UI - Admin	Login as Admin	Insert/Delete visible	Admin controls rendered	PASS
12	Role-based UI - Student	Login as Student	Only Borrow shown	Student view correct	PASS

All 12 test cases passed. No critical bugs found during testing.

13 Conclusion & Future Enhancements

Conclusion

This project successfully demonstrates a complete, end-to-end relational database application. The **Library Circulation and Digital Resource Management System** was designed from first principles using Entity-Relationship modelling, normalized to 3NF across 13 tables, implemented in MySQL 9.7 with InnoDB foreign-key enforcement, and exposed via a Flask REST API connected to a responsive browser-based UI.

The system supports all required CRUD operations (Insert, Delete, View) and executes five analytical SQL queries covering the full operational scope of a university library. The role-based interface cleanly separates Admin and Student workflows.

Future Enhancements

Enhancement	Description
Barcode / RFID Integration	Scan physical accession barcodes for faster issue/return
Email / SMS Notifications	Automated reminders for due dates and fines
Online Payment Gateway	Pay fines directly through the portal
Multi-Branch Support	Branch-specific copy allocation
Advanced Search	Full-text search on title, author, ISBN with filters
Analytics Dashboard	Charts for borrowing trends and peak usage
Mobile Application	React Native / Flutter for student self-service
Cloud Deployment	Deploy on AWS/GCP with managed MySQL (RDS/Cloud SQL)

14 References

1. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill.
2. Ramakrishnan, R., & Gehrke, J. (2002). *Database Management Systems* (3rd ed.). McGraw-Hill.
3. MySQL 9.7 Reference Manual. Oracle Corporation.
<https://dev.mysql.com/doc/refman/9.0/en/>
4. Flask Documentation. Pallets Projects. <https://flask.palletsprojects.com/>
5. PyMySQL Documentation. <https://pymysql.readthedocs.io/en/latest/>
6. MDN Web Docs - HTML, CSS, JavaScript Reference. Mozilla Foundation.
<https://developer.mozilla.org/>
7. Google Fonts - Inter, Space Grotesk, JetBrains Mono. <https://fonts.google.com/>

The complete source code, SQL scripts, ER diagram, presentation slides, and this project report are available in the public GitHub repository below.

Item	Details
Repository Name	DBMS-PROJECT
Student	Rohan Shikhar (25WU0101112)
Repository URL	https://github.com/rohandev0185/DBMS-PROJECT
Folder: Presentation-I	Problem Description slides / PDF
Folder: Presentation-II	Slides, ER diagram, library_db.sql, query solution
Folder: Presentation-III	Flask app source code, templates, static assets, screenshots
Folder: Project-Report	This report (Project_Report.pdf)
README.md	Project overview, roll number, description, setup instructions

Repository Structure

```
DBMS-PROJECT/  
+-- README.md  
+-- Presentation-I/  
|   +-- Problem_Description.html  
+-- Presentation-II/  
|   +-- library_db.sql  
|   +-- er_diagram.png  
+-- Presentation-III/  
|   +-- app.py  
|   +-- config.py  
|   +-- db.py  
|   +-- templates/index.html  
|   +-- static/  
|       +-- css/style.css  
|       +-- js/app.js  
+-- Project-Report/  
    +-- Project_Report.pdf
```